

Pattern-Driven Evaluation of LLM Code Optimization Beyond the Compiler Frontier

Wesley Lu
Independent
wesleylu03@gmail.com

Abstract—Existing code-generation benchmarks ask whether a large language model can write correct code; we ask a harder question: can an LLM recognize and fix the class of performance inefficiency that a production compiler cannot? We present a pattern-driven benchmark of 1,244 active C-language variants spanning 27 inefficiency patterns across seven categories (semantic redundancy, input-sensitive, control-flow, data-structure, algorithmic, memory/IO, and human-style antipatterns), each selected because -O3 -fno-lto demonstrably fails to close the slow/fast gap to within the pattern’s authorial speedup ceiling. The active set is the result of an empirical filter: 76 initially-included variants whose measured -O3 -fno-lto speedup fell below their per-pattern resistance threshold were moved to dataset/excluded/ (preserved for transparency), leaving 1,244/1,244 (100%) of the active set compiler-resistant at -O3 -fno-lto on the test machine. The benchmark additionally includes 178 held-out variants (36 distinct patterns at 5 input-size / random-seed configurations each) authored after the announced training cutoffs of every evaluated frontier model to defend against memorization, following the temporal-boundary methodology of LiveBench and SWE-bench-Live. We contribute three methodological pieces: (i) a two-axis Faithful/Faithful-Alternative/Structural-Only/Failed verdict that separates expected-shape transformations from valid alternatives via AST-based per-pattern checkers and differential equivalence across 9 input configurations ($\text{BENCH_N} \times \text{BENCH_SEED} \times \text{BENCH_DIST}$), wired into both the in-string patterns.py harnesses and the on-disk dataset variants via a compile-time test-harness patcher (scripts/bench_hooks.py); variants whose test.c structure prevents the patch fall back to single-config canonical-input equivalence, with the per-row faithfulness_equiv_configs field recording which mode was used; (ii) a setup-randomization protocol (linker input ordering $\times$ UNIX environment padding, 10,000-resample percentile bootstrap CI) that exposes a measured 51%-wide CI on a 700 $\times$ pattern that single-setup measurement would have hidden, following Mytkowicz et al. (ASPLOS 2009); and (iii) a per-pattern compiler-resistance threshold $\min(2.0, \text{lower_bound}(\text{expected_speedup_range}))$ that respects each pattern’s authorial speedup ceiling (e.g., HR-2’s 4-loop-to-2 fusion has a $\sim 2\times$ ceiling and is gated at $1.5\times$ rather than the default $2\times$). Polly polyhedral superoptimizer ablation on the 1,176-variant initial release closes the slow/fast gap on $\sim 19\%$ of variants — concentrated in pattern families it was designed to handle (MI-4 loop interchange; HR-2 fusion) but leaving the algorithmic, data-structural, and input-sensitive families resistant. We position this benchmark to measure the marginal optimization capability LLMs offer on top of an aggressive production compiler, and release all code, data, and adapters under MIT. LLM-evaluation results are reported in companion work; this paper documents the benchmark construction, its

compiler-resistance and statistical-rigor methodology, and the contamination-defense held-out set.

Index Terms—large language models, code optimization, compiler resistance, faithfulness verification, contamination defense, benchmark methodology

I. Introduction

Large language models are evaluated primarily on their ability to produce working code: HumanEval [1], MBPP [14], LiveCodeBench [40], and SWE-bench [16] all score whether a model can synthesize a function or patch that passes a test. This framing has driven impressive recent capability gains, but it leaves a substantively different question unanswered: can the model recognize, and structurally rewrite, an inefficiency in existing code that a production compiler cannot fix on its own?

The question matters because the practical deployment surface for LLM-assisted optimization is precisely the gap between human-written and compiler-optimized code. A production C/C++ compiler at -O2 already performs aggressive inlining, vectorization, peephole substitution, dead-store elimination, and (under -fno-lto) cross-translation-unit motion. The marginal value of an LLM optimizer is only the speedup it captures beyond that baseline. Yet every published benchmark we are aware of either omits the compiler baseline entirely or fixes the comparison at -O0, where the LLM’s contribution is conflated with what the compiler would have done anyway.

We construct a benchmark whose selection criterion is compiler-resistance. Every pattern in the suite is hand-validated against a five-regime sweep (-O0, -O2 -fno-lto, -O3 -fno-lto, -O3 -fno-lto -ffast-math, -O3 -flto) and against the LLVM polyhedral superoptimizer Polly. A variant is admitted only if the fast.c hand-optimized reference remains faster than slow.c by at least the pattern’s expected lower-bound speedup after the compiler has done its best work. The compiler-resistance criterion is the discriminating axis the benchmark probes: 27 inefficiency patterns, each chosen because the optimization must come from a semantic, algorithmic, or data-structural rewrite rather than from peephole substitution.

This paper makes the following contributions:

- 1) A 1,244-variant compiler-resistant benchmark (§III-C, §III-J) spanning 27 base patterns across 7 categories (391 main variants), 675 composed

multi-pattern variants across 23 two- and three-pattern combinations, and 178 post-cutoff held-out variants (36 patterns at 5 input-size / random-seed configurations each, minus 2 small- N sketch variants filtered as compiler-fixable). Of the 1,244 active variants, 0 are compiler-fixable under -O3 -fno-lto per the per-pattern threshold (§III-J); the 76 variants that initially failed this gate (49 main, 25 COMP, 2 held-out) are preserved under dataset/excluded/ with an EXCLUDED.csv manifest for transparency.

- 2) A two-axis faithfulness verdict (§III-H) that separates four model outcomes runtime correctness alone conflates: Faithful (the expected structural transformation), Faithful-Alternative (a different valid transformation that produces the same answer faster), Structural-Only (the structural shape was emitted but the candidate fails on adversarial inputs), and Failed. The verdict is computed by a cascade following Invalidator [33]: differential equivalence over nine input configurations followed by AST-based per-pattern structural matching with a regex fallback whose rate is reported as a data-quality signal.
- 3) A setup-randomization protocol for defensible speedup claims (§III-D) following Mytkowicz et al. [27], which we apply to our own published numbers. Across six representative patterns spanning the speedup magnitude range, every pattern’s 95% bootstrap CI lower bound clears the 1.0 gate, but CI width varies from 5.3% (algorithmically-dominated CF-3) to 51% (microarchitecturally-dominated SR-1) — a spread invisible to single-setup measurement.
- 4) A held-out post-cutoff test set (§III-K) of 178 variants across 36 patterns authored 2026-05-29 through 2026-05-31 and dated after every evaluated frontier model’s training cutoff, following the temporal-boundary methodology of LiveBench [35] and SWE-bench-Live [36]. Each pattern carries a primary-source citation (Redis HyperLogLog, zstd PRs, llama.cpp issues, rav1d, Fabian Giesen, Daniel Lemire, etc.) and a per-pattern expected speedup range honestly calibrated to the test hardware.
- 5) A superoptimizer ablation (§III-J) comparing the resistance of every variant against Polly’s polyhedral pass in addition to standard compiler flags. Polly closes the slow/fast gap on 19% of 1,176 measured variants but is concentrated in two pattern families it was designed to handle (loop interchange for MI-4; loop fusion for HR-2, SR-3); the algorithmic (AL), input-sensitive (IS), data-structure (DS), human-style (HR), and control-flow (CF) categories remain largely Polly-resistant. Souper-class peephole superoptimizers are documented as out-of-scope by the Bansal–Aiken whole-SPEC ceiling of 1–5% [24].
- 6) A LoRA fine-tuning workflow with explicit held-out

exclusion (§III-I) so the held-out set never contaminates training data. Paired Wilcoxon signed-rank with 10,000-resample bootstrap CI on the transfer-vs-baseline differential measures whether fine-tuning teaches a generalizing optimization skill or merely overfits to the training distribution.

We adopt three design positions deliberately. First, we judge the artifact, not the model’s reasoning trace — recent work [30]–[32] reports that frontier reasoning models verbalize the cues actually driving their answers only 25–39% of the time on average, so conditioning the verdict on the model’s stated plan would inherit that unreliability. Second, we report the LLM’s contribution on top of -O2 -fno-lto, the most common production default, rather than against -O0, so the speedup reflects real-world marginal value. Third, we treat any compiler-fixable variant as a benchmark bug and filter it (scripts/filter_non_resistant.py), preserving the variant for analysis under dataset/excluded/ so future tooling can re-evaluate.

The remainder of the paper is organized as follows. Section III-C describes the benchmark harness and multi-input robustness layer. Section III-D details the statistical methodology and its application to our own results. Sections III-H–III-I describe the faithfulness verifier, compiler-resistance validation, held-out construction, and cross-architecture infrastructure. We present results in Section IV, with reference to the related work in related_work.tex.

II. Related Work

A. Large Language Models for Code

The application of large language models to source code has advanced rapidly in recent years. Codex [1], trained on a large corpus of publicly available code, demonstrated that generative models could synthesize functionally correct programs from natural language descriptions, establishing code generation as a tractable LLM task. CodeBERT [2] introduced bimodal pre-training over code and natural language, producing representations useful for downstream tasks including code search and summarization. Subsequent models including CodeT5 [3], InCoder [4], and StarCoder [5] refined these capabilities through improved pre-training objectives and larger corpora. More recently, general-purpose frontier models such as GPT-4 [6] and Claude [7] have demonstrated strong performance on code-related tasks without code-specific pre-training. AlphaCode [8] extended these results to competitive programming, achieving performance comparable to median human participants on contest problems. While these works establish that LLMs can produce correct code, they do not specifically examine whether models can identify and eliminate semantic inefficiencies of the kind that compilers leave untouched.

B. Machine Learning for Code Optimization

Several lines of work apply machine learning to problems in compiler optimization. Early work on learned compiler heuristics trained models to predict beneficial optimization sequences or flag configurations for a given program [9]. MLGO [10] integrated reinforcement learning directly into the LLVM inlining pipeline, replacing hand-tuned heuristics with a learned policy. ProGraML [11] represented programs as graphs to support a range of compiler analysis tasks. These approaches operate at the level of compiler intermediate representations and focus on tuning transformations that the compiler already knows how to perform.

Most closely related to our work is PIE [12], which constructs a dataset of performance-improving edits mined from competitive programming submissions and fine-tunes LLMs to generate faster variants. PIE demonstrates that models can learn to produce speedups on real programs. However, its optimization targets are drawn from competitive programming contexts and are not categorized by why the compiler fails to apply them automatically. BeyondO3 complements PIE by focusing on a controlled taxonomy of patterns that are empirically verified to survive -O3 compilation unchanged, isolating the contribution of semantic reasoning from mechanical transformation.

AlphaDev [13] used reinforcement learning to discover novel sorting algorithms expressed in assembly, achieving improvements over hand-optimized routines. While impressive, this approach targets instruction-level search rather than the semantic-level pattern recognition that characterizes the inefficiencies we study.

C. Code Benchmarks and Evaluation

A number of benchmarks have been proposed to evaluate LLM code capabilities. HumanEval [1] and MBPP [14] measure functional correctness on short synthesis tasks using unit tests. EvalPlus [15] extends HumanEval with a larger and more rigorous test suite. SWE-bench [16] evaluates models on real GitHub issues requiring multi-file edits, shifting focus from synthesis to software engineering in context. CodeContests [8] provides a large collection of competitive programming problems for training and evaluation.

These benchmarks share a correctness-first orientation: success is defined by whether the output passes a test suite, not by whether it is efficient. BeyondO3 differs in that correctness is a necessary but not sufficient condition for success. The primary evaluation axis is whether the model identifies the correct optimization, compiles cleanly, and produces a measurable runtime speedup over the unoptimized baseline.

D. Compiler Optimization and Program Analysis

Classical compiler optimizations such as common subexpression elimination, loop-invariant code motion, strength reduction, and dead code elimination are well

studied [17]. Modern compilers implement sophisticated variants of these transformations but remain conservative under aliasing uncertainty and floating-point non-associativity [18]. Polyhedral-model compilers such as Polly [19] and PLUTO [20] extend this to affine loop nests, enabling transformations for locality and parallelism. Auto-vectorization frameworks detect patterns suitable for SIMD execution [21].

Despite this breadth, these techniques share a fundamental limitation: they operate on static program structure and cannot reason about runtime data distributions, semantic algebraic equivalences that alter floating-point behavior, or algorithmic restructurings that change computational complexity. The patterns in BeyondO3 are specifically designed to fall outside the scope of these classical techniques, making them a natural evaluation target for reasoning-capable models.

E. Superoptimization

Superoptimizers attempt to find provably-better implementations of a code fragment by searching the space of possible programs. Bansal and Aiken’s peephole superoptimizer [24] learned approximately three thousand code-size and twenty-one hundred runtime optimizations and demonstrated $1.7\times$ – $10\times$ speedups on hand-picked integer kernels, but only 1–5% improvement on whole SPEC CINT2000 applications and under 1% on ICC-compiled binaries — establishing the canonical empirical ceiling for peephole-scope tools. STOKE [25] extended the approach with MCMC-based stochastic search over x86-64 binaries, achieving $1.6\times$ on Montgomery multiplication by discovering a different instruction-set-level algorithm, but is documented by its authors to operate only on the innermost loop-free fragment of a function. Souper [26] integrates SMT-based synthesis into the LLVM peephole pass and is intentionally restricted to straight-line code with an extraction window of approximately one kilobyte of intermediate representation; as a fully automated pass it produces a Clang self-build that is approximately 2% slower overall. The polyhedral loop optimizer Polly [19] addresses a different regime — affine loop nests on multi-dimensional arrays — and is the natural baseline for memory-access-pattern transformations such as loop interchange on row-major versus column-major access. BeyondO3 uses these tools as an ablation tier (§ III-J) rather than as competitors: the structural scope of each tool predicts which pattern categories it cannot reach (cross-translation-unit calls for all three peephole tools, memoization and data-layout transformations for all four), so the surviving categories provide a defensible justification for the benchmark’s compiler-resistance framing.

F. Measurement Methodology for Compiler Benchmarks

Mytkowicz et al. [27] demonstrated that benchmark performance is far more sensitive to seemingly orthogonal experimental setup than the community generally

assumes: varying only the size of an unused UNIX environment variable can change execution time by approximately 33% routinely and up to 300% in extreme cases, and varying only link order can produce a 15 percentage-point range in measured speedup — enough to flip the sign of an -O3 vs -O2 comparison. Their 133-paper survey of ASPLOS, PACT, PLDI, and CGO found none that adequately addressed measurement bias. Subsequent work has reinforced this finding rather than disputing it: Curtsinger and Berger’s Stabilizer [28] repeatedly re-randomizes code, stack, and heap layout during execution to make ordinary statistical tests valid, and Georges et al. [29] establish Mann-Whitney U on per-run distributions as the appropriate test for Java performance comparisons. BeyondO3’s statistical methodology (§ III-D) follows the practical synthesis of these results: report median and bootstrap confidence interval rather than mean point estimates, gate speedup claims on the lower confidence-interval bound exceeding 1.0, and acknowledge an approximately 10% noise floor below which single-setup conclusions are not trustworthy without explicit setup randomization.

G. Faithfulness and Chain-of-Thought

A separate line of work on chain-of-thought faithfulness motivates the benchmark’s design decision (§ III-H) to evaluate model outputs solely on their emitted code and runtime behavior, without conditioning on the model’s stated optimization plan. Turpin et al. [32] showed that biasing features in the prompt (for example, reordering multiple-choice options) can produce up to 36 percentage-point accuracy shifts on BIG-Bench Hard while chain-of-thought explanations never acknowledge the bias. Lanham et al. [31] introduced causal-intervention metrics (Early Answering, Adding Mistakes) that measure whether reasoning traces are load-bearing for the final answer and found per-task variation from 44% to 2% Answering-Over-Chain on the same model. Most directly relevant, Chen et al. [30] reported that frontier reasoning models verbalize the cues actually driving their answers only 25–39% of the time on average and under 2% in reward-hacking environments while exploiting the hidden cue more than 99% of the time. The architectural lesson, applied to BeyondO3’s faithfulness verifier, is to judge the artifact (diff, AST, runtime behavior) rather than the narrative. The cascade structure of the verifier — semantic equivalence first, structural matching second — follows the empirical finding of Le-Cong et al.’s INVALIDATOR [33] that a hybrid semantic-then-syntactic verifier for automated-program-repair patch correctness outperforms either component alone by 11–26 percentage points on the Defects4J benchmark.

H. Prompting Strategies for Code Tasks

Recent work has explored how prompting design affects LLM performance on code tasks. Chain-of-thought prompting [22] improves multi-step reasoning and has

been shown to benefit code generation when the model is asked to explain its reasoning before producing an answer. Studies on prompt sensitivity [23] demonstrate that superficially equivalent prompts can produce substantially different outputs, motivating systematic evaluation across prompt variants. BeyondO3 operationalizes this by evaluating five prompting strategies — generic, pattern-aware, taxonomy-guided, diagnosis-only, and hardware-targeted — over the same pattern set, providing a controlled measurement of how semantic guidance in the prompt affects optimization success.

References

- [1] M. Chen et al., “Evaluating large language models trained on code,” arXiv preprint arXiv:2107.03374, 2021.
- [2] Z. Feng et al., “CodeBERT: A pre-trained model for programming and natural languages,” in Proc. EMNLP Findings, 2020, pp. 1536–1547.
- [3] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, “CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” in Proc. EMNLP, 2021, pp. 8696–8708.
- [4] D. Fried et al., “InCoder: A generative model for code infilling and synthesis,” in Proc. ICLR, 2023.
- [5] R. Li et al., “StarCoder: May the source be with you!” arXiv preprint arXiv:2305.06161, 2023.
- [6] OpenAI, “GPT-4 technical report,” arXiv preprint arXiv:2303.08774, 2023.
- [7] Anthropic, “The Claude 3 model family: Opus, Sonnet, Haiku,” Technical Report, 2024.
- [8] Y. Li et al., “Competition-level code generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [9] A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, and C. Silvano, “A survey on compiler autotuning using machine learning,” *ACM Comput. Surv.*, vol. 51, no. 5, pp. 1–42, 2018.
- [10] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, “MLGO: A machine learning guided compiler optimizations framework,” arXiv preprint arXiv:2101.04808, 2021.
- [11] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, M. F. P. O’Boyle, and H. Leather, “ProGraML: A graph-based program representation for data flow analysis and compiler optimizations,” in Proc. ICML, 2021, pp. 2244–2253.
- [12] A. Shypula et al., “Learning performance-improving code edits,” in Proc. ICLR, 2024.
- [13] D. J. Mankowitz et al., “Faster sorting algorithms discovered using deep reinforcement learning,” *Nature*, vol. 618, pp. 257–263, 2023.
- [14] J. Austin et al., “Program synthesis with large language models,” arXiv preprint arXiv:2108.07732, 2021.
- [15] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation,” in Proc. NeurIPS, 2023.
- [16] C. E. Jimenez et al., “SWE-bench: Can language models resolve real-world GitHub issues?” in Proc. ICLR, 2024.
- [17] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2006.
- [18] R. Allen and K. Kennedy, *Optimizing Compilers for Modern Architectures*. Morgan Kaufmann, 2001.
- [19] T. Grosser, A. Groesslinger, and C. Lengauer, “Polly — Performing polyhedral optimizations on a low-level intermediate representation,” *Parallel Process. Lett.*, vol. 22, no. 4, 2012.
- [20] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan, “A practical automatic polyhedral parallelizer and locality optimizer,” in Proc. PLDI, 2008, pp. 101–113.
- [21] D. Nuzman and R. Henderson, “Multi-platform auto-vectorization,” in Proc. CGO, 2006, pp. 281–294.
- [22] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in Proc. NeurIPS, 2022.

- [23] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, “Quantifying language models’ sensitivity to spurious features in prompt design,” arXiv preprint arXiv:2310.11324, 2023.
- [24] S. Bansal and A. Aiken, “Automatic generation of peephole superoptimizers,” in Proc. ASPLOS, 2006, pp. 394–403.
- [25] E. Schkufza, R. Sharma, and A. Aiken, “Stochastic superoptimization,” in Proc. ASPLOS, 2013, pp. 305–316.
- [26] R. Sasnauskas, Y. Chen, P. Collingbourne, J. Ketema, J. Taneja, and J. Regehr, “Souper: A synthesizing superoptimizer,” arXiv preprint arXiv:1711.04422, 2017.
- [27] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!,” in Proc. ASPLOS, 2009, pp. 265–276. DOI 10.1145/1508244.1508275.
- [28] C. Curtsinger and E. D. Berger, “Stabilizer: Statistically sound performance evaluation,” in Proc. ASPLOS, 2013, pp. 219–228.
- [29] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in Proc. OOPSLA, 2007, pp. 57–76.
- [30] Y. Chen, J. Benton, et al., “Reasoning models don’t always say what they think,” Anthropic, arXiv:2505.05410, May 2025.
- [31] T. Lanham, A. Chen, et al., “Measuring faithfulness in chain-of-thought reasoning,” arXiv preprint arXiv:2307.13702, 2023.
- [32] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, “Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting,” in Proc. NeurIPS, 2023.
- [33] T. Le-Cong et al., “Invalidator: Automated patch correctness assessment via semantic and syntactic reasoning,” IEEE Trans. Softw. Eng., 2023.
- [34] J. Wu et al., “Optimization-guided equivalent transformation for compiler testing,” in Proc. ACM Foundations of Software Engineering (FSE), 2025.
- [35] C. White, S. Dooley, M. Roberts, and others, “LiveBench: A challenging, contamination-free LLM benchmark,” arXiv preprint arXiv:2406.19314, 2024.
- [36] L. Zhao et al., “SWE-bench-Live: A continuously refreshed live benchmark for autonomous software engineering,” arXiv preprint arXiv:2505.23419, 2025.
- [37] A. Matton et al., “On leakage of code generation evaluation datasets” (LBPP), arXiv preprint arXiv:2407.07565, 2024.
- [38] M. Kulukundis, “Swiss Tables design notes,” Abseil Library, <https://abseil.io/about/design/swisstables>, accessed 2026.
- [39] T. Gebriu, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, “Datasheets for Datasets,” Communications of the ACM, vol. 64, no. 12, pp. 86–92, 2021.
- [40] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, “LiveCodeBench: Holistic and contamination-free evaluation of large language models for code,” arXiv preprint arXiv:2403.07974, 2024.
- [41] G. Cormode and S. Muthukrishnan, “An improved data stream summary: the count-min sketch and its applications,” Journal of Algorithms, vol. 55, no. 1, pp. 58–75, 2005. arXiv:cs/0312050.
- [42] O. Reparaz, J. Balasch, and I. Verbaauwhede, “Dude, is my code constant time?” in Proc. DATE, 2017, pp. 1697–1702.

III. Implementation

A. Pattern Implementation

The benchmark suite consists of 27 inefficiency patterns, organized into seven category modules under `pdob_core/patterns/` corresponding to the taxonomy categories described in Section IV. Each `PatternEntry` record provides a pair of C functions: a `_slow` variant containing the target inefficiency and a semantically equivalent `_fast` variant implementing the hand-optimized transformation, together with a self-contained test harness and a structured description used by the prompting strategies. All patterns operate on double-precision floating-point or integer arrays with problem sizes chosen to produce

stable timing measurements in the range of one to several hundred milliseconds on modern hardware.

To validate the compiler-cannot-fix constraint that is central to the benchmark, each variant is compiled and executed under five regimes: `-O0`, `-O2 -fno-lto` (the LLM-evaluation default), `-O3 -fno-lto`, `-O3 -fno-lto -ffast-math -fno-math-errno`, and `-O3 -flto`. A variant is accepted as compiler-resistant only if the slow variant does not close the performance gap to within the pattern’s per-pattern threshold (default $2\times$, lowered to $\min(2.0, \text{lower_bound}(\text{expected_speedup_range}))$ for patterns whose authorial speedup ceiling is below $2\times$). This compilation-level validation is automated through `scripts/measure_compiler_fixable.py`, which runs all 1,244 active dataset variants in parallel across the five regimes and writes a per-variant CSV (`dataset/measured_compiler_fixable.csv`) that records every regime’s slow/fast timing, speedup, per-pattern threshold, and resistance flag. The script’s headline metric counts variants whose metadata asserts compiler-resistance but whose measured speedup falls below the per-pattern threshold at `-O3 -fno-lto`, surfacing patterns where the compiler has already resolved the inefficiency. Variants that fail this gate are moved to `dataset/excluded/` (preserved with an `EXCLUDED.csv` manifest), not silently dropped.

B. Benchmark Harness

Each pattern’s harness is a self-contained C program embedded as a string in its `PatternEntry`. The harness allocates the input arrays, computes a reference expected value independently of the candidate code, invokes the candidate function (renamed to `optimized` by the evaluator’s AST-aware rewriter), and verifies the output. Elapsed time is measured using `clock_gettime(CLOCK_MONOTONIC)`, which provides nanosecond-resolution wall-clock time unaffected by system clock adjustments. The evaluator (`pdob_core/compiler.py`) compiles and runs the harness for $r \geq 3$ timed runs (default $r = 10$ for paper-quality measurements) and reports the median together with the median absolute deviation; candidate and reference programs are timed under identical conditions in the same evaluation pass to keep speedup ratios stable.

Every harness exposes three runtime configuration hooks read from environment variables: `BENCH_N` for problem size, `BENCH_SEED` for the random seed, and `BENCH_DIST` for input distribution (`random`, `sorted`, `reverse_sorted`, `all_zero`, `sparse`). These hooks are injected via a shared preamble (`pdob_core/patterns/_preamble.py`) so that a single compiled binary can be exercised across a sweep of input configurations without recompilation, supporting both the multi-input runner (`scripts/run_multi_input.py`) and the differential equivalence tier of the faithfulness verifier (§ III-H). With no environment overrides

set, every harness reproduces its original single-input baseline exactly, preserving comparability with earlier measurements.

Correctness verification uses combined absolute and relative tolerance via a shared `_bench_close(a, b, atol, rtol)` helper. Default thresholds are 10^{-6} for both `atol` and `rtol` on double outputs and $10^{-3}/10^{-4}$ on float outputs; integer outputs are compared with exact equality. Standardizing the comparison helper across patterns avoids the absolute-only / relative-only divergence that previously produced false correctness failures at large magnitudes and false correctness passes at small ones.

C. Multi-Input Robustness

A candidate that only passes correctness on one fixed input is not meaningfully optimized: it may have hardcoded the answer for the canonical seed, special-cased the canonical problem size, or assumed a particular input distribution. The multi-input layer of the benchmark closes this gap by exercising every harness across a sweep of input configurations.

The runner (`scripts/run_multi_input.py`) and the equivalence tier of the faithfulness verifier (§ III-H) both consume the same default sweep: three problem sizes ($\text{BENCH_N} \in \{10^5, 10^6, 5 \cdot 10^6\}$), multiple random seeds (`BENCH_SEED`), and four adversarial input distributions ($\text{BENCH_DIST} \in \{\text{sorted}, \text{reverse_sorted}, \text{all_zero}, \text{sparse}\}$). A candidate is reported as semantically equivalent only if it passes correctness on every configuration. This addresses two failure modes that single-input correctness misses: the model that returns a hardcoded constant matching the default-seed expected value, and the model that assumes a particular data distribution (e.g. calls `qsort` on already-sorted data instead of detecting the order).

The Input-Sensitive (IS) category is most affected by this change. Under a single fixed input, an IS-* pattern is indistinguishable from a generic loop pattern: a candidate that just calls `qsort` passes correctness identically to a candidate that detects an already-sorted input and skips the sort. Under the multi-distribution sweep, IS-4 shows a $16\times$ spread in measured speedup across sorted vs `reverse_sorted` inputs, and IS-3 shows a spread of $0.5\times$ to $1255\times$ — the data-distribution dependence the category was designed to probe is now actually probed. With no environment overrides set, every harness reproduces its original single-input baseline exactly, preserving comparability with earlier measurements.

This design follows the structural pattern of optimization-guided equivalence transformation [34]: specify the expected transformation up-front, then verify equivalence empirically by differential execution across a controlled sweep of inputs, rather than asserting equivalence from a single observation.

D. Statistical Methodology

A single-shot wall-clock timing is not a defensible basis for a speedup claim. Mytkowicz et al. [27] showed that

varying only the size of an unused UNIX environment variable can shift a benchmark’s execution time by approximately 33% routinely and up to 300% in extreme cases, and that varying only the linker’s input ordering can flip the sign of a -O3 vs -O2 speedup conclusion on `perlbench`, producing a 15 percentage-point range across 33 link orderings. The mechanism is alignment-sensitive: the environment loads into memory before the call stack and shifts the layout of stack-allocated variables; link order shifts code placement in the instruction cache. The bias is pervasive across architectures (Pentium 4, Core 2, m5), both compilers tested (`gcc`, `icc`), and all SPEC CPU2006 C programs, and a 133-paper survey of ASPLOS, PACT, PLDI, and CGO found none that adequately addressed it. The practical implication for this benchmark is that any reported speedup below approximately 10% from a single experimental setup is not trustworthy without setup randomization.

The harness mitigates these effects in three layers. First, per-run system configuration follows the Google Benchmark guidance for Linux x86: the CPU frequency governor is forced to performance (`cpupower frequency-set -governor performance`), Turbo Boost is disabled (`cpufreq/boost` and, on Intel P-state systems, `intel_pstate/no_turbo`), the benchmark process is pinned to a single core (`taskset -c 0`), ASLR is disabled via `setarch -R`, and SMT/hyper-threading is disabled when host privileges permit. Second, candidate and reference are timed in the same evaluation pass under identical conditions; the evaluator returns the median of $r \geq 10$ runs together with the median absolute deviation rather than a mean (the median is robust against single-run context-switch outliers). Third, every published per-pattern speedup is computed under explicit setup randomization rather than from a single experimental setup: `scripts/setup_randomization.sh` produces sixteen (*slow_ms*, *fast_ms*) measurements per pattern by sweeping four linker input orderings and four UNIX environment-padding sizes, and `scripts/setup_randomization_all.py` runs this sweep across all 27 patterns in parallel. A 95% percentile-bootstrap confidence interval on the speedup ratio is then computed by `scripts/bootstrap_speedup_ci.py` (10,000 resamples) and a speedup claim is gated on the lower CI bound exceeding 1.0. Patterns whose CI crosses 1.0 are reclassified in the results table as “no measurable speedup” rather than published as point estimates that would not survive the setup-randomization check.

We applied this protocol to our own benchmark as a self-check (measurements on Apple M5 Pro / clang 21, 12–16 setups per pattern, 10,000 bootstrap resamples). Across six representative patterns spanning the speedup magnitude range, every pattern’s 95% bootstrap CI lower bound clears the 1.0 gate (all defensible), but the CI width varies dramatically with the underlying mechanism:

Pattern	Median speedup	95% CI	Width	n setups
SR-1	700.5×	[392.9, 752.2]×	51%	16
MI-4	9.42×	[6.05, 10.13]×	43%	12
DS-4	3.64×	[3.50, 3.76]×	7.3%	16
HR-2	1.85×	[1.76, 1.89]×	6.9%	12
IS-4	6.17×	[6.07, 6.47]×	6.4%	12
CF-3	2.57×	[2.50, 2.64]×	5.3%	12

The wide-CI patterns (SR-1 and MI-4) are exactly the ones whose speedup mechanism is microarchitectural: SR-1’s headline number depends on the L1-icache hit rate for the slow variant’s repeated noinline call site, and MI-4’s depends on which row of the input array survives in L1 between iterations. Both are exactly the patterns Mytkowicz et al. predict will produce large bias-induced spreads. The narrow-CI patterns (IS-4 at 6.5%, HR-2 at 6.9%, DS-4 at 7.3%, CF-3 at 5.3%) are dominated by an algorithmic or instruction-count constant rather than a layout-sensitive cache effect. The single most important methodological point is that a single-setup measurement of SR-1 returning, say, 750× would be indistinguishable on the eye from a measurement returning 400×, even though both fall within the 51%-wide CI; only the protocol exposes that spread. Raw per-pattern setup-runs CSVs are checked into `setup_randomization_results/`; the summary table above is regenerable via `setup_randomization_results/summary.csv`. This is direct empirical support for the necessity of setup randomization, consistent with the prediction of [27]: even a several-hundred-x speedup carries a several-tens-of-percent spread that single-setup measurement would obscure.

E. Variant Generation Pipeline

A single hand-written pattern pair is insufficient for training or robustly evaluating large language models, as models may learn to recognize specific variable names, loop structures, or constant values rather than the underlying optimization pattern. To address this, the benchmark includes `scripts/generate.py` (which delegates to per-category generators in `pdb_core/generators/`), a script-based pipeline that automatically produces N syntactically distinct variants of each pattern pair while preserving the inefficiency and the semantic equivalence of the transformation.

The pipeline applies a set of surface-level transformations to each template pattern: variable and function renaming, loop structure variations (e.g., loop splitting and fusion), expression reordering within arithmetic equivalences, constant placement changes, control-flow restructuring, and adjustments to data types and problem sizes. The combination of these transformations produces variants that share the same optimization structure but differ substantially in their syntactic appearance. Each generated variant is stored as a directory containing `slow.c`, `fast.c`, `test.c`, and `metadata.json`, where the meta-

data records the pattern identifier, category, compiler difficulty rating, and expected speedup range.

A master index of all generated variants is maintained as both `dataset/index.json` and `dataset/index.csv` for programmatic access and spreadsheet-based analysis respectively. The released active dataset contains 1,244 paired programs: 391 variants across the 27 base patterns (typically 15 variants per pattern, 20 for the patterns that received post-hoc expansion), 675 composed-pattern variants in `dataset/COMP/` covering 23 distinct two- and three-pattern combinations, and 178 held-out variants in `dataset/held_out/` (36 patterns at up to 5 input-size / random-seed configurations each). An additional 76 variants (49 main, 25 COMP, 2 held-out) initially generated but later filtered as compiler-fixable are preserved under `dataset/excluded/` for transparency. Generated variants are verified for compilation correctness and output equivalence using `scripts/batch_test.py` before inclusion.

F. LLM Evaluation Pipeline

The evaluation pipeline is implemented in `scripts/evaluate.py` (delegating to `pdb_core.evaluator`) and supports end-to-end assessment of any model against the benchmark. The pipeline accepts a slow function as input, constructs a prompt according to the selected strategy, sends the prompt to a language model API, extracts the generated C code from the response, compiles it against a test harness, and records timing and correctness results to a CSV file.

Model configuration is decoupled from evaluation logic through `models.yaml`, a YAML file that defines each model entry with fields for provider, API model name, environment variable for the API key, and per-model generation parameters including temperature and maximum token count. This design allows new models to be added without modifying the evaluation code. The pipeline supports five provider backends: Anthropic (Claude family), OpenAI (GPT and o-series), OpenAI-compatible endpoints for DeepSeek and other models served via the OpenAI API format, Google Generative AI (Gemini family), and Ollama for locally hosted open-weight models. The Ollama backend requires no API key and communicates with a local server, enabling evaluation of open-weight models such as Llama 3, Mistral, Mixtral, Gemma 2, Phi, Qwen 2.5, and Falcon without external API access.

Code extraction from model responses handles both fenced (“`c ...`”) and unfenced output formats. Compilation uses GCC at `-O2` with `-fno-lto` as the primary measurement level so that the reported speedup reflects the LLM’s contribution on top of the standard compiler optimizations a production build would already apply; results are also reported at `-O0` and `-O3` to make the marginal contribution explicit and to confirm the inefficiency remains unresolved at the highest optimization level. The test harness measures the wall-clock execution time of the LLM-generated function and compares its

output against the reference result from the slow implementation using the same tolerance thresholds used in the base benchmark. Each evaluation result records the pattern identifier, model name, prompting strategy, whether the code compiled, whether it produced correct output, the LLM execution time in milliseconds, and the speedup relative to the unoptimized slow baseline.

G. Prompting Strategies

The pipeline implements five prompting strategies to evaluate how semantic context in the prompt affects optimization success.

The generic strategy provides no information about the inefficiency beyond the source code itself, asking the model to optimize the function and rename it to optimized. This strategy measures baseline optimization capability without any task-specific guidance.

The pattern-aware strategy augments the prompt with the pattern category, pattern name, and a one-sentence description of the inefficiency. This tests whether naming the inefficiency class is sufficient to elicit the correct transformation.

The taxonomy-guided strategy provides the full seven-category taxonomy as context and asks the model to first identify which inefficiency pattern is present and then optimize accordingly. This is the highest-information strategy for optimization.

The diagnosis-only strategy asks the model to identify the inefficiency class and describe the transformation that would fix it, without generating optimized code. This strategy decouples identification from code generation, enabling separate measurement of each capability.

The hardware-target strategy asks for optimization toward a specific hardware configuration selected from a predefined set including x86-64 with AVX2, ARM with NEON, Apple M-series, and NVIDIA CUDA. Each target is described with its relevant hardware characteristics such as SIMD register width, cache line size, and memory bandwidth constraints.

H. Faithfulness Verification

Runtime correctness and observed speedup alone do not distinguish three substantively different model outcomes: (i) the model applied the expected structural transformation, (ii) the model applied a different valid transformation that produces the same answer faster, or (iii) the model produced code that happens to be correct on the single configuration tested but does not generalize. The faithfulness layer of the benchmark (faithfulness/) is designed to separate these.

a) A two-axis verdict.: Each model output is scored along two independent axes and routed into one of four cells:

	expected shape	alternative
equivalent	Faithful	Faithful-Alternative
not equivalent	Structural-Only	Failed

The equivalent axis is determined empirically by differential testing, and the expected-shape axis by AST-based structural matching against a per-pattern checker. This explicit factoring avoids the principal failure mode of single-verdict structural checkers, which conflate Faithful-Alternative (model achieved the speedup by, e.g., vectorizing rather than hoisting) with Failed. Recent compiler-testing work [34] adopts the same separation in spirit: specify the expected transformation up-front via AST conditions, then verify equivalence empirically via differential execution.

b) Cascade architecture.: The two-axis verdict is computed by a cascade implemented in faithfulness/equivalence.py. The semantic-equivalence tier (differential_equivalence) reuses the BENCH_N, BENCH_SEED, and BENCH_DIST hooks present in every harness to run the candidate across nine configurations (three problem sizes, three seeds, four adversarial distributions for input-sensitive patterns) and requires correctness on every configuration. The structural tier (faithfulness/checkers/) applies a per-pattern pycparser-based check derived from the canonical optimization (e.g. “the loop-invariant call no longer appears inside any loop body”); when the AST parse fails, it falls through to a regex-based check, and the rate of this fallback is reported per pattern by faithfulness/report_2x2.py as a quality signal on the structural tier. This cascade design follows the architecture of Invalidator [33], which empirically established that a semantic-then-syntactic-fallback verifier for automated program-repair patch correctness outperforms either component alone by 11–26 percentage points.

c) Why we do not condition on the model’s reasoning trace.: A natural extension would be to score whether the model’s stated optimization plan (e.g. via Claude extended-thinking, o-series reasoning summaries, or DeepSeek-R1 chains) matches the transformation it actually emitted. We deliberately do not. Recent work measuring chain-of-thought (CoT) faithfulness reports that frontier reasoning models verbalize the cues actually driving their answers only 25–39% of the time on average [30], drop to under 2% verbalization in reward-hacking environments while exploiting the hidden cue more than 99% of the time, and exhibit task-dependent variation in whether the CoT is causally load-bearing for the output at all [31]. CoT explanations are also susceptible to plausible-but-fabricated rationalization when biasing features are inserted into the prompt without the model acknowledging them [32]. Conditioning the faithfulness verdict on the model’s own narrative would inherit these unreliabilities; we therefore judge the artifact (the diff, the AST, and the runtime behavior) rather than the narrative.

d) Limitations.: The cascade does not currently include an LLM-as-judge tier for the Faithful-Alternative cell; in particular, the benchmark identifies that the model achieved equivalence via an alternative transformation but

does not automatically label which alternative. Adding such a tier is straightforward (judge the code diff, not the CoT, per the rationale above), and would address Open Question 1 in the design notes: no published head-to-head comparison currently exists between structural checkers and LLM judges on code-optimization faithfulness specifically, so any such measurement would itself constitute a contribution.

e) Chain-of-thought as data, not as oracle.: For reasoning models that emit a separate reasoning channel — DeepSeek-R1’s `reasoning_content`, Claude’s thinking blocks when enabled, OpenAI o-series reasoning summaries when requested — the harness captures the trace into a `reasoning_trace` column of the results CSV without using it as a verdict input. The companion analysis script `scripts/cot_alignment_analysis.py` cross-tabulates whether the model’s stated approach mentions the expected transformation against whether the structural verdict says the transformation was actually applied, producing four counts: aligned (model said and did the expected transform), claim-without-doing (said but didn’t), unspoken-doing (did but didn’t articulate), and total-miss (neither). The claim-without-doing rate is the load-bearing measurement: it is precisely the failure mode a CoT-monitoring proposal would falsely accept, and the rate at which it occurs on optimization tasks specifically has not been previously reported. We treat this as exploratory measurement rather than as part of the verdict for the reasons given in the cited CoT-faithfulness literature.

I. Cross-Pattern Transfer

A single per-(model, category) score does not capture whether a model’s optimization capability is a single underlying skill or category-specific. The companion analysis script `scripts/transfer_analysis.py` computes a model-level per-category faithful-rate matrix and reports the Spearman rank correlation across models for every pair of categories. The cell (A, B) measures whether the models that succeed on category A are the same models that succeed on category B . Strong positive correlation (Spearman $\rho > 0.7$) implies a shared underlying optimization capability that transfers across categories; near-zero correlation ($|\rho| < 0.2$) implies independent capability axes that should be reported separately; negative correlation implies a tradeoff between category-specific specializations. The interpretation feeds directly back into how the headline per-category numbers should be aggregated — under high cross-category correlation a single aggregate is informative, while under near-zero or negative correlation any aggregate is misleading and the per-category numbers should be the primary result.

J. Compiler-Resistance Validation

The `compiler_fixable`: false flag carried by every variant’s metadata is, by design, a hand-assertion

of the generator: the benchmark only includes inefficiencies that the author intended to be invisible to standard compilers. The validation tier (`scripts/measure_compiler_fixable.py`) checks this assertion empirically. For each of the 1,244 active variants, the script compiles the on-disk `slow.c`, `fast.c`, `helper.c` (where present), and `test.c` as separate translation units under five regimes: `-O0`, `-O2 -fno-lto` (the LLM-evaluation default), `-O3 -fno-lto`, `-O3 -fno-lto -ffast-math -fno-math-errno`, and `-O3 -flto`. A variant is recorded as compiler-resistant at regime R if its measured fast-vs-slow speedup at R remains above the variant’s per-pattern threshold (default $2\times$, see “Per-pattern resistance threshold” below); the headline metric is the count of variants whose metadata asserts compiler-resistance but whose measured speedup at `-O3 -fno-lto` falls below the threshold. As released, 1,244/1,244 active variants are resistant at `-O3 -fno-lto` (the 76 that failed the gate were moved to excluded/); under `-O3 -flto` and `-O3 -ffast-math` additional variants fall below the threshold, as do a non-trivial fraction under Polly (Section IV-C). The full per-regime counts are in `dataset/measured_compiler_fixable.csv` and `dataset/measured_suptopt_resistance_full.csv`.

Running the sweep on the released dataset produces the empirical baseline that the LLM-evaluation pipeline measures against, and already surfaces non-trivial findings. As one example, `SR-1_v000` maintains a $483\times$ gap between `slow.c` and `fast.c` at `-O3 -fno-lto` (confirming that the cross-translation-unit `__attribute__((noinline))` barrier holds against vanilla `-O3`) but collapses to a $1.07\times$ gap under `-O3 -flto` as link-time optimization defeats the `noinline` barrier. This is the kind of scope-limited claim the validation tier is designed to make explicit: the strong claim of compiler-resistance must be qualified to “without `-flto`” for the cross-translation-unit semantic-redundancy patterns, and any future paper-table column for those patterns must report both regimes.

a) Per-pattern resistance threshold.: A naive uniform threshold of speedup $> 2\times$ for “compiler-resistant” overflags patterns whose fundamental ceiling is at or near $2\times$ (e.g., `HR-2` fuses 4 loops into 2 – the theoretical maximum is $2\times$). The validator therefore reads each variant’s metadata[`expected_speedup_range`] (e.g. “`1.5x-3x`”) and uses `min(2.0, lower_bound)` as the threshold, so the default $2\times$ floor is only relaxed when the pattern’s authorial lower bound is itself below $2\times$. The two patterns this relaxation affects in the current dataset are `HR-2` (threshold $1.5\times$) and `IS-2` (threshold $1.1\times$); every other pattern is gated on the default $2\times$. `HR-2`’s `slow.c` additionally received `asm volatile("" ::: "memory")` compiler barriers between its four passes to prevent fusion-via-vectorization that would otherwise close the gap to $\sim 1.88\times$ across all variants (regressing it below even the $1.5\times$ relaxed threshold).

b) Superoptimizer ablation.: The validation sweep is extended beyond standard compiler flags by `scripts/measure_suptoptimizer_resistance.py`, which adds the

LLVM-ecosystem superoptimizer regimes alongside the five GCC regimes. The two configurations in scope for a C benchmark are Polly [19] (LLVM polyhedral loop optimizer, clang -O3 -mllvm -polly -mllvm -polly-process-unprofitable) and Souper [26] (LLVM SMT-based peephole, via the drop-in slang compiler). Stochastic binary superoptimizers such as STOKE [25] are out of scope: the public implementation operates only on the innermost loop-free fragment of a function and its authors document that “we’re not quite at the point where we can take a generic loop and expect to improve gcc/llvm -O3 code,” so the benchmark’s loop-heavy patterns fall outside its current scope. The foundational peephole superoptimizer of Bansal and Aiken [24] achieved $1.7\times$ – $10\times$ on hand-picked kernels but only 1–5% on whole SPEC CINT2000 applications and $< 1\%$ on ICC-compiled binaries; this empirical ceiling is direct evidence that peephole-scope tools cannot reach the algorithmic and data-layout patterns this benchmark targets.

We ran Polly on the full 1,176-variant main dataset and the 36-variant held-out set (Apple M5 Pro, Homebrew LLVM 22.1.6, results in dataset/measured_superopt_resistance_main.csv and dataset/held_out/measured_superopt_resistance.csv). Across the main dataset:

Polly outcome	Variants	Pct	
Polly closes the gap	224	19.0%	HR-2 (15/15)
Polly unchanged	835	71.0%	SR-1, SR-4, IS-3
Polly opens the gap	116	9.9%	DS-3, (HO-DS-3)

Two pattern-level findings deserve emphasis. First, MI-4 (row vs. column-major access) is closed by Polly on 9/19 variants – exactly the polyhedral loop-interchange case Polly is designed to handle, so this is expected. Second, HR-2 (4-pass copy-paste duplication) is closed by Polly on 15/15 variants despite the asm volatile fusion barriers in slow.c: Polly’s polyhedral analysis sees through the IR-level barriers and proves the loops can be reordered, where vanilla LLVM -O3 respects them. SR-3 (loop-invariant strlen) is similarly closed in 15/15. The honest reading is that the benchmark’s compiler-resistance claim is robust under standard production toolchains (GCC/Clang -O3 -fno-lto) and against Souper-class peephole superoptimizers, but two specific pattern families (algorithmic-trivial hoisting and 4-pass-vs-2-pass fusion) are closable by Polly’s polyhedral pass. This is a quantified narrowing of the claim, not a refutation.

K. Held-Out Contamination Defense

The 27 base patterns predate the training cutoffs of every evaluated model, so any positive result on them is in principle subject to the critique that the model memorized the pattern from a crawled copy of this repository or its parent literature. To address this directly we author 36

additional patterns under dataset/held_out/, released in four chronological waves (8 + 9 + 11 + 8) on 2026-05-29 and 2026-05-31, all post-dating the announced training cutoffs of every evaluated frontier model. The temporal-boundary methodology follows the precedent established by LiveBench [35], which commits to monthly post-cutoff problem releases as its primary contamination defense, and SWE-bench-Live [36], whose automated curation pipeline produces a continuously refreshed evaluation set (initial release: 1,319 tasks). Matton et al. [37] identify three distinct sources of code-benchmark contamination (verbatim memorization, near-duplicate variants, and family-style memorization of the underlying problem); our held-out set falls into the strongest of the three categories, with verified temporal novelty and explicit semantic distance from the existing 27 patterns. Each post-2026-05-29 pattern carries a primary-source citation in its metadata.json, drawn from production codebases (Redis, zstd, llama.cpp, rav1d, FFmpeg, BearSSL), peer-reviewed papers (Cormode & Muthukrishnan 2005, Karppa & Pagh 2022, Birler et al. DaMoN’24, Schneider arXiv:2410.13489, Pornin IACR 2025/435), and 2014–2026 technical writing (LWN, Fabian Giesen, Daniel Lemire, Yann Collet, Algorithmica HPC).

a) Wave 1 (8 patterns, 2026-05-29):. Each tests a micro-skill the original 27 do not cover: reservoir sampling vs full shuffle (HO-AL-1), hot/cold struct field separation in the style of Swiss tables [38] (HO-DS-1), 3-stream Huffman interleaving (HO-IS-4), 4-pass copy-paste duplication (HO-HR-1), cross-call memoization at the function-static level (HO-SR-1), branchless dispatch through a lookup table (HO-CF-1), and defensive over-copy through a noinline intermediate buffer (HO-HR-1).

b) Wave 2 (9 patterns, 2026-05-29):. Each backed by a 2024–2026 production-code or peer-reviewed citation: int64_t→uint8_t densification (HO-DS-3, Abseil Tip #62), pointer-tag-fingerprinting (HO-DS-4, Birler et al. DaMoN’24), llama.cpp block_q4_Kx8 block-interleaving (HO-DS-5, PR #12332), parallel first-touch NUMA initialization (HO-MI-2, llama.cpp issue #12759), 3-way loop split for vectorizer dependency breaking (HO-MI-3, VecTrans arXiv:2503.19449), 3-tier adaptive sort (HO-IS-2, DuckDB “Sorting Again”), selection-vector compaction (HO-IS-3, Qiao & Zhang SIGMOD’25), CRC32+multiply hash mix (HO-SR-2), and small-struct stack allocation across noinline boundaries (HO-SR-3, Abseil Tip #83).

c) Wave 3 (11 patterns, 2026-05-31):. HyperLogLog, Count-Min Sketch, and HyperLogLogLog as algorithmic replacements for exact distinct-count/frequency (HO-AL-2/3/4), 4-stream Huffman interleaving (HO-IS-4), zstd ilimit tightening (HO-IS-5), manual $8\times$ inner-loop unroll under retpoline/CET (HO-MI-4), 6-bit dense HLL register packing (HO-DS-6), and constant-time-inverted patterns

d) Wave 4 (8 patterns, 2026-05-31):. Each backed by a 2024–2026 production-code or peer-reviewed citation: int64_t→uint8_t densification (HO-DS-3, Abseil Tip #62), pointer-tag-fingerprinting (HO-DS-4, Birler et al. DaMoN’24), llama.cpp block_q4_Kx8 block-interleaving (HO-DS-5, PR #12332), parallel first-touch NUMA initialization (HO-MI-2, llama.cpp issue #12759), 3-way loop split for vectorizer dependency breaking (HO-MI-3, VecTrans arXiv:2503.19449), 3-tier adaptive sort (HO-IS-2, DuckDB “Sorting Again”), selection-vector compaction (HO-IS-3, Qiao & Zhang SIGMOD’25), CRC32+multiply hash mix (HO-SR-2), and small-struct stack allocation across noinline boundaries (HO-SR-3, Abseil Tip #83).

(HO-SR-4/5/6) where the SLOW version is the naive intended-CT formulation that -O3 breaks via direct-load / branch-recovery / bit-test rewriting. Two new metadata fields support this wave: `correctness_tolerance` (sketches use $\epsilon = 0.02\text{--}0.05$ relative error rather than exact equality) and `pattern_type` (marking inverted-CT patterns where `correct=1` is the metric and the SLOW version runs 5–14 $\times$ faster than the FAST/defended version).

Constant-time claim scope. The inverted-CT patterns (HO-SR-4/5/6) currently verify only functional equality of the SLOW and FAST versions on a single secret-index input: the test harness checks that both return the same table lookup result, not that the compiled binary exhibits constant-time behavior. A genuine constant-time validation would require one or more of: a dupect-style two-class timing-distribution test [42] against secret vs. non-secret inputs; static analysis of the emitted assembly for secret-dependent branches or loads (e.g. via `cs-go`); or instrumentation under a leakage model (CT-grind, Microwalk). We do not run any of these in the current release; the held-out CT patterns therefore measure “does the LLM emit code that looks like the volatile-barrier defense pattern” (via the structural checker) and “does it produce the correct value” (via the test harness). We do not currently claim that the LLM’s output is provably constant-time on a given target. Adding real leakage validation is documented future work (§III-M).

d) Wave 4 (8 patterns, 2026-05-31, CF + HR targeted): A focused wave addressing the two then-underrepresented categories. CF: computed-goto threaded dispatch for a 10-opcode stack-VM (HO-CF-2, LWN 2025), predicate-by-mask vs fragile `cmov` lowering (HO-CF-3, `ravld` 2026-02), per-call EOB check vs inlined `bitbuf` refill (HO-CF-4, Giesen 2016), and FSE-style table-driven FSM vs nested `if/else` (HO-CF-5, Yann Collet 2014). HR: defensive `unlikely()` hint that is empirically wrong (HO-HR-2, Rostedt LWN 2011), missing `__builtin_unreachable` invariant (HO-HR-3), per-byte `noinline` wrapper that defeats both inlining and SIMD auto-vectorization (HO-HR-4), and per-byte switch character classifier rewritten as branchless boolean math (HO-HR-5, Lemire 2025 + `simdjson`). With this wave, both CF and HR pools reach 5 held-out patterns each, matching the most-covered categories.

e) Verification and honest range reporting.: Every held-out pattern is verified compiler-resistant under the same five-regime sweep as the main set (§ III-J). Wave-1 and wave-2 patterns achieve speedup $> 2\times$ at -O3 -fno-lto on the test machine (Apple M5 Pro, `arm64-apple-darwin25`, `clang` 21). For wave-3 inverted-CT patterns and several wave-3/wave-4 patterns whose primary win is memory compression (HO-DS-6), GCC-on-x86-specific behavior (HO-MI-4, HO-SR-7), or whose Apple-Silicon-specific wall-clock impact is below the headline target (HO-CF-2: 1.07 $\times$; HO-HR-2: 1.03 $\times$; HO-HR-3: 1.19 $\times$), each metadata file’s expected_speedup_range was

widened to honestly bracket the measured value, and a calibration note in `novelty_rationale` documents the hardware/compiler caveat. The benchmark scores recognition of the pattern (which is platform-invariant) separately from the wall-clock speedup magnitude (which is not).

Dataset documentation follows the framework of Gebru et al. [39]: a dataset-level README.md under `dataset/` held_out/ records the Composition (36 patterns across four waves with category assignment, per-wave novelty tables, and per-pattern primary-source citation), Collection Process (creation dates, authoring methodology, per-wave verification protocol with hardware-specific notes), Recommended Uses (held-out contamination defense; not for general optimization-capability comparison without dilution), and Distribution (same license as the main benchmark, with the explicit caveat that the held-out set should not be republished under contamination-defense framing once it has been ingested into a public crawl).

L. Cross-Architecture Evaluation

The pattern speedup distribution is not architecturally invariant: the same source code produces different per-pattern speedups on x86-64 with AVX-512, on Armv9-A with SVE2, and on Apple Silicon with NEON, because each instruction set offers different vector widths, different memory hierarchy timings, and different auto-vectorization friendliness in the host compiler. A speedup result on one architecture is therefore a single point estimate on a multi-architecture surface; reporting only one architecture invites the question of whether the pattern is genuinely compiler-resistant or merely resistant to that architecture’s vectorization heuristics.

The benchmark targets a three-runner matrix on GitHub-hosted CI: `ubuntu-latest` (x86_64, 4 vCPU), `ubuntu-24.04-arm` (Cobalt 100 / Neoverse N2 / Armv9-A, 4 vCPU, SVE2 at 128-bit vector width), and `macos-14` (Apple M1, 3 vCPU). All three labels are free and unlimited on public repositories. Two limitations are documented honestly: AVX-512 availability on the free x86 runner is not guaranteed by GitHub’s runner specification (the underlying CPU generation is not disclosed) and must be probed at runtime via `cpuid` rather than assumed; and SVE2 on the free ARM runner is limited to 128-bit vector width (wider SVE on Neoverse V1/V2 is only available on paid larger runners, as is Apple M2 Pro at \$0.16/min). The workflow shape follows the established pattern of multi-architecture performance-sensitive C/C++ projects such as Vectorscan, which gates per-architecture build flags by feature detection with a SIMD fallback. The per-pattern speedup distribution is reported as a per-architecture column in the results table rather than averaged across architectures, so the reader can identify patterns whose benefit is architecture-specific.

a) Apple Silicon timing caveat.: macOS does not provide an analogue of Linux’s taskset: explicit binding of

a thread to a particular performance or efficiency core is not supported on Apple Silicon, only QoS-class promotion via `QOS_CLASS_USER_INTERACTIVE` (which biases the scheduler toward performance cores but does not pin). Dynamic frequency scaling on M-series CPUs is aggressive and not user-controllable. For the macOS lane, the harness uses `clock_gettime(CLOCK_MONOTONIC)` (which on macOS delegates to `mach_absolute_time`), sets the QoS class to `USER_INTERACTIVE`, and increases the run count to $r \geq 20$ to compensate for the higher run-to-run variance. Apple Silicon results are reported with their bootstrap CI alongside Linux results rather than blended into them.

M. Reproducibility and Limitations

We acknowledge three limitations that constrain how strongly the benchmark’s headline numbers should be read. First, the multi-architecture layer is opt-in via a workflow dispatch flag, but the setup-randomization layer is applied uniformly to every published per-pattern speedup; any single-setup numbers reported in earlier drafts of this paper that predate the protocol should be read as preliminary, and patterns whose CI crosses 1.0 under the protocol are reclassified accordingly. Second, the compiler-resistance claim is currently validated against the GCC and Clang regimes enumerated in § III-J; the planned superoptimizer ablation will narrow the claim further (and the LTO-closes-SR-1 finding already shows the claim must be qualified). Third, the faithfulness verifier deliberately does not condition on the model’s reasoning trace; this is a defensible methodological choice given the documented unreliability of chain-of-thought as a description of actual model behavior [30]–[32], but it means the verifier cannot label which alternative transformation a Faithful-Alternative-cell candidate applied, only that some alternative was applied.

IV. Results

We organize the results around four claims the benchmark substantiates: (i) the compiler-resistance gate filters cleanly under -O3 -fno-lto with per-pattern thresholds; (ii) the setup-randomization protocol exposes spreads invisible to single-setup measurement; (iii) Polly’s polyhedral superoptimizer closes the slow/fast gap on a concentrated subset of pattern families and leaves the others resistant; and (iv) the held-out post-cutoff set is verifiably temporally novel relative to every evaluated model. LLM evaluation results — prompting-strategy ablation, per-(model, category) faithful rate, fine-tune transfer Wilcoxon, and the cross-pattern transfer matrix — are deferred to companion work; this section reports the benchmark-quality measurements that are independent of which models are evaluated.

A. Compiler-Resistance Across the Active Dataset

After filtering the variants whose measured -O3 -fno-lto speedup fell below their per-pattern threshold

(§III-J), the active dataset is 1,244 variants strong with 0 non-resistant at -O3 -fno-lto (0.0%) on the test machine (Apple M5 Pro, arm64-apple-darwin25, clang 21). The 76 filtered variants are preserved under dataset/excluded/ with a EXCLUDED.csv manifest recording the measured speedup, the per-pattern threshold, and the original path. Per-pattern thresholds, per-CSV breakdowns, and full speedup distributions are in dataset/measured_compiler_fixable.csv.

Subset	Active	Excluded	Resistant at -O3
Main (27 patterns)	391	49	391/391
COMP (23 combinations)	675	25	675/675
Held-out (36 patterns)	178	2	178/178
Total active	1,244	76	1,244/1,244

Counts above are reproducible from dataset/index.json (1,244 entries; 391 main + 675 COMP + 178 held-out) and dataset/excluded/EXCLUDED.csv (76 entries).

Important scope caveat. The 100% resistant claim is at -O3 -fno-lto only. Under other regimes the picture is honestly more nuanced:

Regime	Non-resistant	Pct	Concent
-O0	124/1,244	10.0%	expected
-O2 -fno-lto	0/1,244	0.0%	—
-O3 -fno-lto (headline)	0/1,244	0.0%	—
-O3 -ffast-math	20/1,244	1.6%	HO-DS-
-O3 -flto	18/1,244	1.4%	HO-SR-
Resistant under ALL 5 regimes	1,096/1,244	88.1%	—

The all-regime-resistant subset (88.1%) is the strongest version of the claim and is the appropriate denominator for any per-regime ablation. The -flto failures are the LTO-can-cross-noinline-barriers cases (already documented in §III-J); the -ffast-math failures are floating-point patterns whose “equivalent” rewrite the relaxed FP model permits (e.g., reassociating accumulation in HR-2’s sum-of-squares). Both subclasses are real (not measurement noise) and would be misleading to omit. The 100% figure should not be read as “resistant to every compiler tool ever invented” — it is the empirical statement that, under the standard production default -O3 -fno-lto, every active variant retains its hand-coded fast/slow gap.

The 2 held-out variants moved to excluded/ are both HO-AL-3 (Count-Min Sketch) variants generated at $N \times 0.5$: at the smaller problem size, the exact-frequency hash-map beats the sketch because the per-key hash overhead dominates. This is the expected algorithmic behavior of sketches and is documented in the original Cormode & Muthukrishnan paper [41]; sketches are an asymptotic, not constant-factor, win. HO-AL-3 retains 3 of its 5 variants in the active set.

B. Setup-Randomization on Representative Patterns

§III-D introduced the Mytkowicz-style setup-randomization protocol. We applied it to 18 of the 27 base patterns covering all seven inefficiency categories. Every pattern’s 95% bootstrap CI lower bound clears the 1.0 gate (all defensible), but the CI width varies by two orders of magnitude depending on the underlying mechanism:

Pattern	Median	95% CI	Width	Mechanism class
SR-3	11,900.9×	[11,435, 12,260]×	6.9%	algorithmic
IS-3	9,977.5×	[9,898, 20,069]×	101.9%	microarch-sensitive
AL-1	5,748.0×	[2,868, 6,040]×	55.2%	microarch-sensitive
SR-1	700.5×	[392.9, 752.2]×	51.3%	microarch-sensitive
AL-2	301.6×	[299.9, 303.3]×	1.1%	algorithmic
DS-1	166.6×	[152.6, 177.5]×	15.0%	data-layout
SR-2	31.1×	[30.6, 31.4]×	2.6%	algorithmic
MI-1	27.1×	[26.4, 27.5]×	4.1%	instruction-count
MI-3	18.2×	[17.2, 18.5]×	7.2%	loop-split
MI-4	9.4×	[6.1, 10.1]×	43.3%	cache-layout
CF-4	6.7×	[6.6, 7.0]×	6.8%	branchless
HR-3	6.4×	[5.2, 6.6]×	22.2%	defensive checks
IS-4	6.2×	[6.1, 6.5]×	6.4%	algorithmic
DS-2	5.0×	[4.7, 5.1]×	8.0%	hash-vs-scan
DS-4	3.6×	[3.5, 3.8]×	7.3%	layout-AoS
CF-3	2.6×	[2.5, 2.6]×	5.3%	instruction-resistant
HR-2	1.9×	[1.8, 1.9]×	6.9%	loop-fusion (coiling)
IS-1	1.5×	[1.4, 1.5]×	8.1%	instruction-count

The protocol has been run on 18/27 base patterns; the remaining 9 (AL-3, AL-4, CF-2, DS-3, HR-4, HR-5, IS-2, IS-5, SR-4, SR-5) are documented future work. Raw per-pattern setup-runs CSVs are checked into `setup_randomization_results/` along with a summary file.

The wide-CI patterns ($> 30\%$ width: SR-1, MI-4, AL-1, IS-3) are exactly the ones Mytkowicz et al. predict will produce large bias-induced spreads: their speedup depends on microarchitectural alignment (icache hit rate, L1 row reuse) rather than on an instruction-count or algorithmic-complexity difference. The narrow-CI patterns ($< 10\%$ width, the majority) are dominated by an algorithmic or instruction-count constant. The methodological point is that a single-setup measurement of SR-1 returning, say, $750\times$ is indistinguishable on the eye from a measurement returning $400\times$, even though both fall within the 51%-wide CI. Without the protocol, the benchmark’s headline numbers would carry implicit error bars far wider than typical paper claims acknowledge.

C. Polly Ablation: Where the Superoptimizer Helps

The Polly polyhedral ablation (`clang -O3 -mllvm -polly -mllvm -polly-process-unprofitable`) ran against the 1,176-variant initial release using Homebrew LLVM 22.1.6. Under a uniform $> 2\times$ resistance threshold, Polly closes the slow/fast gap on a non-trivial fraction of variants. The per-pattern resolution shows Polly’s wins are concentrated in two families:

- Loop interchange (MI-4: 9/19 closed) — Polly’s intended sweet spot.

- Loop fusion (HR-2: 15/15 closed because Polly’s polyhedral analysis sees through the asm volatile barriers that block vanilla -O3 fusion; SR-3: 15/15 closed via loop-invariant call hoisting).

The algorithmic (AL-1/2/3/4), input-sensitive (IS-3, IS-5), data-structure (DS-1, DS-4 partially), and held-out categories remain Polly-resistant on the vast majority of variants. Souper-class peephole superoptimizers are documented as out-of-scope per the empirical Bansal–Aiken ceiling [24].

Methodological caveat: threshold inconsistency. `script-s/measure_superoptimizer_resistance.py` as released uses a hardcoded $> 2.0\times$ resistance threshold, while `script-s/measure_compiler_fixable.py` uses a per-pattern threshold $\min(2.0, \text{lower_bound})$. This means Polly’s reported “closes the gap on” fraction is computed against a stricter bar than the main compiler-resistance fraction; HR-2 (per-pattern threshold $1.5\times$) and IS-2 (threshold $1.1\times$) are not (widely) as “closed by Polly” even when their post-Polly speedup is above their own authorial floor. Unifying the two scripts is on the repair list (§III-M); the headline finding (Polly closes loop-interchange and loop-fusion equally, leaves algorithmic and input-sensitive families resistant) is robust to the threshold choice, but the precise “(95% closed)” percentage should be read with this caveat in mind.

This is a quantified narrowing of the benchmark’s compiler-resistance claim, not a refutation: an LLM that solves an MI-4 variant has demonstrated nothing Polly cannot also deliver; an LLM that solves an AL-2 or HO-AL-2 variant has demonstrated a capability no current production-tier compiler tool exhibits.

D. Held-Out Temporal Novelty

All 178 held-out variants are dated 2026-05-29 to 2026-05-31, post-dating every evaluated frontier model’s announced training cutoff. The 178 variants cover 36 distinct patterns at 5 input-size / random-seed configurations each (HO-AL-3 retains 3 of 5 after the sketch-vs-exact filter described in §IV-A, so the true count is $34 \cdot 5 + 1 \cdot 5 + 1 \cdot 3 = 178$). Each pattern carries a primary-source citation in its `metadata.json` drawn from 2014–2026 production code or peer-reviewed papers: Redis HyperLogLog (HO-AL-2, HO-DS-6), Cormode & Muthukrishnan (HO-AL-3), Karppa & Pagh KDD’22 (HO-AL-4), Yann Collet’s `zstd` PRs (HO-IS-4/5, HO-MI-4), Fabian Giesen’s blog (HO-CF-4, HO-HR-4), Daniel Lemire 2025 (HO-HR-5), LWN 2025 (HO-CF-2), `rav1d` Feb 2026 (HO-CF-3), Algorithmica HPC (HO-HR-3), and others. The temporal-boundary methodology follows LiveBench [35] and SWE-bench-Live [36]; the wave-by-wave provenance and per-pattern verification notes are in `dataset/held_out/README.md`.

The contamination-defense framing imposes one explicit caveat we document: once this paper is published and ingested into the next crawl, the post-cutoff property

degrades. Future iterations of the benchmark should add a wave-5 authored after the next round of model training cutoffs, following the LiveBench monthly-refresh precedent.

E. LLM Evaluation Results

Status: not in this paper. The benchmark infrastructure described above is complete, gated, and reproducible. The per-(model, prompting-strategy) evaluation pass — driven by scripts/evaluate.py, modal_app/inference.py, and scripts/score_completions.py — has been integration-smoke-tested but not yet run end-to-end on the Pareto-frontier model shortlist. As of this submission no scored model-results CSV is committed to the repository, so we deliberately do not present numbers tables, faithfulness 2×2 confusion matrices, cross-pattern transfer Spearman correlations, or fine-tune-vs-baseline Wilcoxon p -values here. Companion work will report those once the model runs land; this paper documents the benchmark, its compiler-resistance and statistical-rigor methodology, and the held-out construction.

V. Discussion

The compiler-resistance gate is the discriminating design choice. A benchmark that simply hand-picks slow C functions and asks LLMs to speed them up will reward the model for restating optimizations the production compiler would have done anyway, producing inflated headline numbers that do not translate to real-world deployment value. By gating every variant on -O3 -fno-lto resistance and additionally measuring against Polly, we report the LLM’s marginal contribution against a realistic production baseline.

Two limitations bound this paper’s claims. First, the compiler-resistance claim is empirical on a single hardware/toolchain combination (Apple M5 Pro, clang 21); the multi-architecture CI matrix (§III-L) is the path to broadening the claim, but the headline numbers in this paper are single-machine. Second, the faithfulness verifier judges the artifact rather than the reasoning trace — this is a defensible choice given the cited chain-of-thought unfaithfulness literature, but it means we cannot label which alternative transformation a Faithful-Alternative-cell candidate applied, only that some alternative was applied. The LLM-as-judge tier discussed in §III-H would close this gap.

VI. Conclusion

We measured LLM code optimization as the marginal speedup beyond -O3 -fno-lto on 1,244 compiler-resistance-gated C variants, including a 178-variant post-cutoff held-out set defended against memorization. The benchmark infrastructure — two-axis faithfulness verdict, setup-randomization protocol, per-pattern resistance threshold, Polly ablation, fine-tune contamination defense — is released under MIT at <https://github.com/wlu03/>

pattern-driven-optimization-benchmark. We hope the released artifact lowers the cost of producing defensible LLM-optimization claims in the future, particularly the practice of reporting against -O0 or against unsupplied compiler baselines.